https: / / github.com / fhlsteven / CLR_via_CSharp / blob / main / chapters / ch17_Delegates.md 我现在在学习《CLR via C#》这本书的第17章，文章内容如链接所示，我们将围绕此进行讨论

用户:

https://github.com/fhlsteven/CLR_via_CSharp/blob/main/chapters/ch17_Delegates.md

我现在在学习《CLR via C#》这本书的第17章，文章内容如链接所示，我们将围绕此进行讨论，我也会向你请教。

元宝:

好，第17章「委托」是《CLR via C#》里非常核心的一章，也是很多人读CLR时会反复回头看的一章——因为它把 C# 表面上那点 `delegate` / `+=` / `=>` 语法糖，一层层剥到 CLR 层面的 `_target`、`_methodPtr`、`_invocationList`，看完会对「事件、lambda 闭包、async 回调」这些上层东西理解深很多。

你贴的这份笔记我扫了一遍，覆盖了原书 17 章的全部小节：

- **17.1–17.3** 委托初识 + 静态/实例方法回调（本质就是 `_target` 为 null 或非 null）
- **17.4 委托揭秘** 最关键——`delegate` 关键字经编译器后会变成一个 **继承自 `MulticastDelegate` 的类**，带 `Invoke` / `BeginInvoke` / `EndInvoke`，这一节吃透后面都顺
- **17.5 委托链** `Delegate.Combine / Remove` + `_invocationList` 那个数组结构，还有 `GetInvocationList` 手动遍历解决"一个抛异常全链断"的问题
- **17.6 泛型委托** `Action / Func` 那一堆，为什么微软后来不建议你再自己 `delegate` 了
- **17.7 C# 简化语法** 不 new 委托对象、lambda、闭包（局部变量被编译器拎进一个 `<>c__DisplayClass` 里，这点在调试和内存方面容易踩坑）
- **17.8 委托+反射** `CreateDelegate + DynamicInvoke`，日常写得少，但看框架代码会遇到

你现在是**刚读到哪一节**，还是已经有具体卡点了？常见的几个容易懵的地方我可以提前说一下方向：

1. `_invocationList` **那个"每次 Combine 都新建一个委托对象"** 的结构——为什么要这样设计而不是直接链表？（涉及不可变+线程安全）
2. **委托链返回值被丢弃**那段伪代码——为什么微软不改？改成返回数组行不行？
3. **lambda 捕获局部变量**那节，编译器生成的 `<>c__DisplayClass` 什么时候会让对象生命周期意外变长（内存泄漏老坑）
4. 和第 11 章「事件」、第 27 章 `BeginInvoke/EndInvoke`（虽然现在 obsolete 了）的衔接

你接下来想顺着章节走，还是直接抛你卡住的那个点？

用户:

这里提到的qsort是什么？

“C “运行时”的 qsort 函数获取指向一个回调函数的指针，以便对数组中的元素进行排序。在 Microsoft Windows 中，窗口过程、钩子过程和异步过程调用等都需要回调函数。在 .NET Framework 中，回调方法的应用更是广泛。例如，可以登记回调方法来获得各种各样的通知，例如未处理的异常、窗口状态变化、菜单项选择、文件系统变化、窗体控件事件和异步操作已完成等。”

元宝:

这里的 `qsort` 指的是 **C 语言标准库里的快速排序函数**，是讲解「**回调函数（Callback）**」最经典、最入门的例子之一，作者拿来和 .NET 里的委托做类比。

我给你拆开说一下，顺便把和 C# 的对应关系讲清楚。

---

1. `qsort` 是什么？

`qsort` 定义在 C 标准库 `<stdlib.h>` 中，原型大致是：

```
    void qsort(
        void *base,           // 要排序的数组起始地址
        size_t nmemb,         // 元素个数
        size_t size,          // 每个元素的大小（字节）
        int (*compar)(const void *, const void *)  // 比较函数指针
    );
```

关键点：
`qsort` 只负责“怎么排”（算法），但**完全不知道你怎么比较两个元素**，所以它要求你传一个 **函数指针**（`compar`），这个函数由你来写。

---

2. 为什么 `qsort` 一定要回调？

因为 `qsort` 是**通用排序算法**：

- 它不知道你排的是 `int[]`、`double[]`、结构体数组，还是字符串数组
- 它也不知道你是要升序还是降序
- 甚至不知道你的排序规则（按年龄？按姓名？）

所以它把“**比较两个元素大小**”这件事**交还给你**，这就是**回调（callback）**。

示例（C 语言）

```c
#include <stdlib.h>
#include <stdio.h>

int compare_ints(const void *a, const void *b) {
    int ia = *(int *)a;
    int ib = *(int *)b;
    return ia - ib;  // 升序
}

int main() {
    int arr[] = {4, 2, 9, 1};
    qsort(arr, 4, sizeof(int), compare_ints);
}
```

这里发生了什么？

1. `qsort` 内部会多次调用 `compare_ints`
2. `qsort` **反过来调用你提供的函数**
3. 这就是“回调”

---

3. 和《CLR via C#》第 17 章的关系

作者在讲 **委托的本质 = 类型安全的函数指针。**

C / C++	.NET / C#
函数指针（`int (*)(void*, void*)`）	委托（`delegate`）
`qsort` 接收比较函数	`Array.Sort` 接收 `Comparison<T>`
回调不可知类型	委托是强类型
无闭包支持	Lambda / 闭包

C# 里的“qsort”

在 .NET 中，等价的东西是：

```c
int[] arr = {4, 2, 9, 1};
Array.Sort(arr, (x, y) => x.CompareTo(y));
```

或者更底层一点：

```c
Array.Sort(arr, new Comparison<int>((x, y) => x - y));
```

👉 `Comparison<int>` **就是 .NET 版的“比较函数指针”**
👉 **委托就是 CLR 对“函数指针 + this + 类型安全”的封装**

---

4. 作者为什么在这里提 `qsort` ？

因为：

- `qsort` 是 **几乎所有程序员第一次接触回调函数的地方**
- 它能非常直观地说明：

> “一个通用算法，通过函数指针把决策权交给调用者”

- 而 **委托 = 托管世界里的“升级版函数指针”**

随后作者顺势引出：

- Windows API 里的窗口过程（ `WndProc` ）
- 钩子（ `HookProc` ）
- APC（异步过程调用）
- .NET 里的事件、通知、异步完成回调

这些都是 **同一个思想的不同形态。**

---

5. 一句话总结

> **`qsort` 是一个用“函数指针做回调”的经典 C 运行时函数，用来排序任意类型的数组；而 .NET 中的委托，就是对这种回调机制的现代化、类型安全、面向对象封装。**

如果你愿意，我也可以帮你把 `qsort` → `Array.Sort` → `IComparer<T>` → `delegate` → `MulticastDelegate` 这条链路串成一张“心智模型图”，对理解第 17 章特别有用。

用户:

文章中的例子不太懂：

using System;

using System.Windows.Forms;

using System.IO;

// 声明一个委托类型，它的实例引用一个方法，

// 该方法获取一个 Int32 参数，返回 void

internal delegate void Feedback(Int32 value);

public sealed class Program {

```
    public static void Main() {
```

```
        StaticDelegateDemo();

        InstanceDelegateDemo();

        ChainDelegateDemo1(new Program());

        ChainDelegateDemo2(new Program());

    }



    private static void StaticDelegateDemo() {

        Console.WriteLine("----- Static Delegate Demo -----");

        Counter(1, 3, null);

        Counter(1, 3, new Feedback(Program.FeedbackToConsole));

        Counter(1, 3, new Feedback(FeedbackToMsgBox));          // 前缀 "Program." 可选

        Console.WriteLine();

    }



    private static void InstanceDelegateDemo() {

        Console.WriteLine("----- Instance Delegate Demo -----");

        Program p = new Program();

        Counter(1, 3, new Feedback(p.FeedbackToFile));

        Console.WriteLine();

    }



    private static void ChainDelegateDemo1(Program p) {

        Console.WriteLine("----- Chain Delegate Demo 1 -----");
```

```
    Feedback fb1 = new Feedback(FeedbackToConsole);
    Feedback fb2 = new Feedback(FeedbackToMsgBox);
    Feedback fb3 = new Feedback(p.FeedbackToFile);

    Feedback fbChain = null;
    fbChain = (Feedback) Delegate.Combine(fbChain, fb1);
    fbChain = (Feedback) Delegate.Combine(fbChain, fb2);
    fbChain = (Feedback) Delegate.Combine(fbChain, fb3);
    Counter(1, 2, fbChain);

    Console.WriteLine();
    fbChain = (Feedback)
        Delegate.Remove(fbChain, new Feedback(FeedbackToMsgBox));
    Counter(1, 2, fbChain);
}

private static void ChainDelegateDemo2(Program p) {
    Console.WriteLine("----- Chain Delegate Demo 2 -----");
    Feedback fb1 = new Feedback(FeedbackToConsole);
    Feedback fb2 = new Feedback(FeedbackToMsgBox);
    Feedback fb3 = new Feedback(p.FeedbackToFile);
```

```
        Feedback fbChain = null;

        fbChain += fb1;

        fbChain += fb2;

        fbChain += fb3;

        Counter(1, 2, fbChain);



        Console.WriteLine();

        fbChain -= new Feedback(FeedbackToMsgBox);

        Counter(1, 2, fbChain);

    }



    private static void Counter(Int32 from, Int32 to, Feedback fb) {

        for (Int32 val = from; val <= to; val++) {

            // 如果指定了任何回调，就调用它们

            if (fb != null)

                fb(val);

        }

    }



    private static void FeedbackToConsole(Int32 value) {

        Console.WriteLine("Item=" + value);

    }
```

```
    private static void FeedbackToMsgBox(Int32 value) {

        MessageBox.Show("Item=" + value);

    }



    private void FeedbackToFile(Int32 value) {

        using (StreamWriter sw = new StreamWriter("Status", true)) {

            sw.WriteLine("Item=" + value);

        }

    }
```

}

元宝:

这个例子是 Jeffrey Richter 讲**委托本质**的“标杆示例”，它把「委托 = 函数指针 + 对象引用 + 类型安全」这件事演得非常完整。我带你**逐段拆开，但不啰嗦**，重点放在“为什么这么写”和“CLR 背后在干什么”。

一、先抓主干：这段代码到底在干嘛？

一句话总结：

> **Counter 负责“数数”，但它不决定“数到之后干什么”；**
> **Feedback 委托负责“接住这个数，去做不同的事”。**

这就是 **回调（Callback）** 的核心思想，也是委托存在的意义。

二、最核心的一行：委托声明

```
    internal delegate void Feedback(Int32 value);
```

这行代码编译后，CLR 会生成一个**类**，等价于：

```
    sealed class Feedback : MulticastDelegate {
        public Feedback(Object target, IntPtr methodPtr) { ... }
        public void Invoke(Int32 value) { ... }
    }
```

记住三个关键字段（CLR via C# 重点）：

字段	含义
`_target`	为 null → 静态方法；非 null → 实例方法
`_methodPtr`	指向方法的 IL 入口
`_invocationList`	多播委托时，指向方法数组

三、**Counter**：委托的“使用者”

```
    private static void Counter(Int32 from, Int32 to, Feedback fb) {
        for (Int32 val = from; val <= to; val++) {
            if (fb != null)
                fb(val);
        }
    }
```

重点理解 **`fb(val)`**

- `fb(val)` 实际调用的是：`fb.Invoke(val)`
- `Counter` **完全不知道**：
  - 是谁的方法
  - 是静态还是实例
  - 是打印、弹框还是写文件
- 它只知道：**“你给我一个 Feedback，我就把 val 丢给你”**

✅ 这就是**解耦**

四、**StaticDelegateDemo**：静态方法回调

```
    Counter(1, 3, new Feedback(Program.FeedbackToConsole));
```

对应 **CLR** 内部：

成员	值
`_target`	`null`（静态方法）
`_methodPtr`	`FeedbackToConsole` 的地址

这两个写法等价：

```
new Feedback(Program.FeedbackToConsole);
new Feedback(FeedbackToConsole); // 同一类内可省略类名
```

为什么第一个 `Counter(1,3,null)` 没反应？

```
if (fb != null)
```

`fb` 为空，什么都不做。

五、**InstanceDelegateDemo**：实例方法回调（⭐重点）

```
Program p = new Program();
Counter(1, 3, new Feedback(p.FeedbackToFile));
```

CLR 内部：

成员	值
`_target`	`p`（这个 Program 实例）
`_methodPtr`	`FeedbackToFile`

关键区别（面试常问）：

类型	_target
静态委托	`null`
实例委托	指向某个对象

👉 **实例方法回调 = 方法 + 调用该方法的对象**

这也是为什么实例方法可以访问 `this`、`sw.WriteLine` 等。

六、**ChainDelegateDemo1**：委托链（多播委托）

构建委托链

```
Feedback fbChain = null;
fbChain = (Feedback)Delegate.Combine(fbChain, fb1);
```

```
fbChain = (Feedback)Delegate.Combine(fbChain, fb2);
fbChain = (Feedback)Delegate.Combine(fbChain, fb3);
```

发生了什么？

- `Delegate.Combine` **不会修改原有委托**
- 而是**创建一个新委托**
- `_invocationList` 变成一个数组：

```
[ FeedbackToConsole,
  FeedbackToMsgBox,
  FeedbackToFile ]
```

调用时：

```
fbChain(1);
```

等价于：

```
FeedbackToConsole(1);
FeedbackToMsgBox(1);
FeedbackToFile(1);
```

⚠️ 注意：**一旦某一个抛异常，后面的都不会执行**

Remove 一个委托

```
Delegate.Remove(fbChain, new Feedback(FeedbackToMsgBox));
```

- 不是“移除同一个对象引用”
- 而是**按方法签名匹配**
- 返回一个新的委托链

七、ChainDelegateDemo2：`+= / -=` 语法糖（⭐重点）

```
fbChain += fb1;
fbChain -= new Feedback(FeedbackToMsgBox);
```

编译器实际做的事：

```
fbChain = (Feedback)Delegate.Combine(fbChain, fb1);
fbChain = (Feedback)Delegate.Remove(fbChain, new Feedback(FeedbackToMsgBox));
```

✅ `+= / -=` **只是语法糖**
✅ 背后仍然是 `Combine / Remove`
✅ 委托是**不可变对象**

八、为什么需要 `new Feedback(...)`？

你可能会问：

> 为什么 Remove 的时候还要 `new Feedback(...)`？

因为：

- 委托比较的是 `_target + _methodPtr`
- 不是比较"同一个变量"
- `new Feedback(FeedbackToMsgBox)` 会生成一个指向同一方法的委托

这在 CLR 层面是**等值匹配**，不是引用匹配。

九、用一个"现实类比"帮你记

把委托想成 **遥控器按钮**：

- `Feedback`：按钮接口（只有一个"按下"动作）
- `FeedbackToConsole`：按下去 → 打印
- `FeedbackToMsgBox`：按下去 → 弹窗
- `FeedbackToFile`：按下去 → 写文件
- `Counter`：只负责"按按钮"
- `+=`：把多个电器接到同一个按钮上

十、你现在应该重点记住的 5 句话

1. **委托是类，继承自** `MulticastDelegate`
2. **静态方法** → `_target == null`
3. **实例方法** → `_target == 对象实例`
4. `+= / -=` **本质是** `Combine / Remove`，**委托不可变**
5. **委托链内部是数组** `_invocationList`

如果你愿意，下一步我可以帮你把这段代码的 **IL / CLR 内部结构画成一张对照表**，或者直接帮你过渡到 **事件（event）为什么要用委托** ——那是第 11 章最容易被这块卡住的地方。你更想继续哪一块？

用户:

和11章的事件"https://github.com/fhlsteven/CLR_via_CSharp/blob/main/chapters/ch11_Events.md"，结合讲解一下；比如事件是方法，委托是类，这样理解对吗？还有InstanceDelegateDemo中new Feedback(p.FeedbackToFile)好像也没对第一个参数_target赋值，这是怎么回事？

元宝:

这是一个**非常高质量的提问**，两个问题都直接命中了 CLR 对「委托 / 事件」建模的核心差异。我们一个一个来，把第 11 章（事件）和第 17 章（委托）彻底串起来。

一、先纠正一个关键理解：「事件是方法，委托是类」❌

你的直觉方向是对的，但表述需要微调，这是很多人学 CLR 时最容易混淆的点。

✅ 正确说法

概念	本质
委托（delegate）	👉 **类**（编译器生成的，继承自 `MulticastDelegate`）
事件（event）	👉 **成员**（像属性 property 一样，是对委托访问的封装）
add / remove	👉 **方法**（事件访问器，类似属性的 get / set）

❌ 错误理解

> “事件是方法”

事件**不是方法**，事件是一个**成员**，它背后**依赖一个私有的委托字段**，并通过一对方法（`add_XXX` / `remove_XXX`）来控制对这个字段的访问。

你可以把事件理解成：

> **事件 = 一个私有委托字段 + 两个公共访问器方法**

二、用 IL 视角重新看事件（第 11 章核心）

你贴的第 11 章里有这一段（非常关键）：

```
    public event EventHandler<NewMailEventArgs> NewMail;
```

C# 编译器会把它“展开”成 **3 个东西**：

```
// 1️⃣ 私有委托字段（真正存回调的地方）
private EventHandler<NewMailEventArgs> NewMail = null;

// 2️⃣ 添加回调的方法
public void add_NewMail(EventHandler<NewMailEventArgs> value) { ... }

// 3️⃣ 移除回调的方法
public void remove_NewMail(EventHandler<NewMailEventArgs> value) { ... }
```

重点来了👇

- **外部代码只能调用 `+=` / `-=`**
- **外部代码永远拿不到 `NewMail` 这个字段本身**
- 只有 `MailManager` 自己能直接访问 `NewMail` 字段并调用它

👉 这就是为什么事件比裸委托"安全"。

三、回到你的第二个问题：`new Feedback(p.FeedbackToFile)` 为什么没看到 `_target`？

这是**编译器帮你"偷偷干了活"**。

1️⃣ 回顾第 17 章的委托本质

你说得完全对：

```
internal delegate void Feedback(Int32 value);
```

编译器会生成这样一个类：

```
sealed class Feedback : MulticastDelegate {
    public Feedback(Object target, IntPtr methodPtr) { ... }
    public void Invoke(Int32 value) { ... }
}
```

其中：

字段	含义
`_target`	实例方法 → 指向对象；静态方法 → null
`_methodPtr`	方法指针

2️⃣ 那么这句代码发生了什么？

```
new Feedback(p.FeedbackToFile)
```

你**没看到** `_target = p`，是因为：

> **这是构造函数参数，不是字段赋值语句**

编译器在编译期把它翻译成：

```
new Feedback(p, typeof(Program).GetMethod("FeedbackToFile"))
```

也就是：

- `p` → 传给构造函数 → 变成 `_target`
- `FeedbackToFile` → 解析成 method handle → 变成 `_methodPtr`

✅ 所以结论是：

> **`_target` 确实被赋值了，只是不是你写的，而是编译器在构造委托对象时完成的。**

3️⃣ 对比静态 vs 实例（非常关键）

```
new Feedback(Program.FeedbackToConsole);   // _target = null
new Feedback(p.FeedbackToFile);             // _target = p
```

- 静态方法：不需要对象 → `_target = null`
- 实例方法：必须知道“哪个对象” → `_target = p`

这也是为什么实例委托能访问 `this`，而静态委托不能。

四、把「事件」和「委托」用一句话连起来（**CLR** 视角）

> **事件是对委托字段的封装；**
> **委托是 CLR 对“方法 + 调用对象”的类型安全封装；**
> **`+=` / `-=` 本质是对委托链的操作。**

用你已有的代码举例

```
mm.NewMail += FaxMsg;
```

编译后等价于：

```
mm.add_NewMail(new EventHandler<NewMailEventArgs>(faxObject, FaxMsg));
```

内部发生的事情是：

1. 创建一个委托对象
2. `_target = faxObject`
3. `_methodPtr = FaxMsg`
4. `Delegate.Combine`
5. 放回私有字段 `NewMail`

五、为什么要有“事件”？直接用 public 委托不行吗？

这是个**面试题级问题**，Richter 在第 11 章专门解释了。

❌ 如果 MailManager 暴露的是 public 委托字段

```
public EventHandler<NewMailEventArgs> NewMail;
```

那么任何人都可以：

- 直接 `NewMail = null`（清空别人注册的回调）
- 直接 `NewMail.Invoke(...)`（乱引发事件）
- 直接覆盖委托链

✅ 事件做了什么？

```
public event EventHandler<NewMailEventArgs> NewMail;
```

只允许：

```
+=   // 添加
-=   // 移除
```

不允许：

```
=      // 禁止赋值
Invoke // 禁止外部触发
```

👉 **事件 = 委托 + 访问控制**

六、一个“遥控器”类比（强烈推荐记住）

- **委托** = 一个遥控器（能按按钮）
- **`_target`** = 被控制的电器

- `_methodPtr` = 按钮对应的功能
- **委托链** = 一个按钮控制一排电器
- **事件** = 遥控器的**安全外壳**
  - 外面只能“加电器 / 删电器”
  - 不能“自己按按钮”
  - 只有厂商（类内部）能按

七、你现在的理解，已经接近 CLR 层面了 ✅

你问的两个问题，其实已经跨过了“语法糖”阶段，进入到了：

- 委托的 `_target` 机制 ✅
- 事件 vs 委托的语义边界 ✅
- 编译器在背后生成了什么 ✅

接下来你可以继续深挖的方向：

1. **为什么事件要用 `EventHandler<TEventArgs>` 而不是自定义委托？**
2. `OnNewMail` **为什么要** `protected virtual` **？**
3. **显式实现事件（EventSet）到底解决了什么性能问题？**
4. `Delegate.Combine` **返回的“新委托”为什么不影响旧的？**

如果你愿意，下一步我可以帮你把 **“事件 → 委托 → _target → 线程安全引发 → EventSet”** 画成一张**完整执行时序图**，这对理解第 11 章是杀手锏级别的。你更想继续哪一个？

用户:

文章中提到“委托对象是方法的包装器(wrapper)，使方法能通过包装器来间接回调。”，这里的包装器是什么？

元宝:

这个问题问得很到位——"包装器"这个词在中文里听着有点虚，但在 CLR 层面它是**非常具体的东西。**

一句话先给结论

> **"包装器" = 那个由 `delegate` 关键字生成的、继承自 `MulticastDelegate` 的类实例。**
> 它里面包着 `_target` + `_methodPtr`，让你"不直接调方法，而是通过这个对象去调方法"。

一、为什么需要"包装"？

C# 里你不能这么写：

```
// ❌ 不可能
void (*callback)(int) = Program.FeedbackToConsole; // C++ 写法，C# 不允许裸函数指针
```

因为：

- C# 是类型安全的
- 方法不是一等公民（不能直接当变量传）
- 实例方法还需要"绑一个对象"才能调

所以编译器必须**把"方法 + 它所属的对象"打包成一个对象**，这个对象就是**包装器。**

二、包装器里到底包了什么？

以你的代码为例：

```
delegate void Feedback(Int32 value);
new Feedback(p.FeedbackToFile);
```

编译器生成的类（简化版）：

```
sealed class Feedback : MulticastDelegate {
    private object _target;      // 👈 包的是"对象"
    private IntPtr _methodPtr;   // 👈 包的是"方法指针"
    private object[] _invocationList; // 👈 多播时包"一串"

    public void Invoke(Int32 value) {
        // 实际调用
    }
}
```

你 `new Feedback(p.FeedbackToFile)` 这一句，**包装器被构造出来时**：

包装的内容	值
`_target`	`p`（Program 实例）
`_methodPtr`	`FeedbackToFile` 的 IL 入口
`_invocationList`	`null`（单播）

三、"间接回调"四个字是什么意思？

直接调用（没有包装器）

```
p.FeedbackToFile(1);  // 直接调，编译期绑定
```

间接调用（有包装器）

```
Feedback fb = new Feedback(p.FeedbackToFile);
fb(1);   // 实际是 fb.Invoke(1)
```

调用路径变成了：

```
fb(1)
 ↓ Invoke
 ↓ 从 _target 拿出 p
 ↓ 从 _methodPtr 拿出 FeedbackToFile
 ↓ p.FeedbackToFile(1)
```

👉 **你调的是"包装器"，包装器再去调"真正的方法"**
👉 这就是"间接"

四、为什么叫 **wrapper** 而不叫 **pointer**？

对比 C：

```
// C：qsort 拿到的是裸指针
void (*compar)(const void*, const void*);
```

C 的指针**只包了函数地址**，没包对象 → 所以 C 排结构体数组时，你得把"上下文"塞进全局变量或参数里硬传。

C# 的委托包装器**包了两样东西**：

- ✅ 方法（像 C 的函数指针）
- ✅ 对象（C 没有，C# 靠 `_target`）

所以 Richter 才说它是 **"方法的包装器"**——不止包方法，还包"这个方法属于谁"。

五、用实例把"包装"两个字钉死

静态方法

```
new Feedback(Program.FeedbackToConsole);
```

包装器内容：

- `_target = null`（没对象可包）
- `_methodPtr = FeedbackToConsole`

→ 包的是"一个静态方法"

实例方法

```
new Feedback(p.FeedbackToFile);
```

包装器内容：

- `_target = p`
- `_methodPtr = FeedbackToFile`

→ 包的是"p 这个对象上的 FeedbackToFile 方法"

多播

```
Feedback fb = fb1 + fb2 + fb3;
```

包装器内容：

- `_invocationList = [fb1, fb2, fb3]`

→ 包的是"一串包装器"

六、一句更直白的翻译

Richter 那句：

> "委托对象是方法的包装器，使方法能通过包装器来间接回调。"

换成人话：

> **委托 = 把"方法 + 它挂在哪个对象上"打包成一个对象，让你能像传变量一样传方法，并且调的时候不用管它是静态的还是实例的。**

如果你愿意，下一步可以把"包装器"这个概念再往下压一层——看 `Invoke` 在 CLR 里到底是怎么通过 `_target` + `_methodPtr` 跳转到 IL 的，那块是 Richter 在 17 章后面"委托揭秘"里讲的，和 JIT 关系很深。要继续吗？

用户:

这一段不太懂，结合例子解释一下：

“如果委托要通过 C#的 params 关键字获取数量可变的参数，要为委托的任何参数指定默认值，或者要对委托的泛型类型参数进行约束，也必须定义自己的委托类型。”

元宝:

这段话其实是在说 **`Action` / `Func` 的“边界”**。它们很好用，但**不是万能的**。一旦你想玩点“高级语法”，就**必须自己定义 delegate**。

我结合你手头的代码，**逐条拆开讲**，你会瞬间明白。

一、先给结论（人话版）

> **`Action` / `Func` 是“最简通用模板”，它们不支持：**
>
> - `params`
> - 参数默认值
> - 泛型约束
>
> 所以只要你有这三种需求之一，**`Action/Func` 就用不了，必须自己写 `delegate`。**

二、背景：你现在用的是哪种委托？

你的代码里是这样声明的：

```
internal delegate void Feedback(Int32 value);
```

这是**你自己定义的委托类型。**

而下面这些是 .NET 自带的“现成委托”：

```
Action<int>              // void (int)
Func<int, string>        // string (int)
Predicate<int>           // bool (int)
```

Richter 的意思是：
👉 **这些现成的委托，功能有限。**

三、情况 1：想用 `params`（最常见）

❌ 不能用 `Action`

```
// 编译错误！
Action<string[]> a = Console.WriteLine; // WriteLine 实际是 params
```

为什么？
因为 `Action<T>` 的定义是：

```
public delegate void Action<T>(T obj);
```

它是**一个参数**，不是 `params T[]`。

✅ 必须自己定义 **delegate**

```
delegate void MyLogger(params string[] messages);

class Program {
    static void Log(params string[] messages) {
        Console.WriteLine(string.Join(", ", messages));
    }

    static void Main() {
        MyLogger log = Log;
        log("a", "b", "c");
    }
}
```

📌 **只有你自己定义的 delegate，才能写成 `params`。**

四、情况 2：想给参数设默认值

❌ `Action` 不支持默认值

```
// ❌ 编译错误
Action<int> a = (x = 10) => { };
```

哪怕你这样写也不行：

```
void Foo(int x = 10) { }
Action<int> a = Foo; // ❌
```

原因很简单：

> **默认值写在“方法定义”上，不是写在“委托签名”上。**

✅ 正确做法：默认值写在方法上

```
delegate void MyDelegate(int x = 10);

class Program {
    static void Foo(int x = 10) {
        Console.WriteLine(x);
    }

    static void Main() {
        MyDelegate d = Foo;
        d();      // 输出 10
        d(20);    // 输出 20
```

```
        }
    }
```

📌 **默认值生效的前提：是你自己定义的 delegate**

五、情况 3：想对泛型参数加约束（⭐重点）

❌ `Func` 没有任何约束

```
Func<T, string> f; // T 是什么？啥都能来
```

但你可能只想让它处理：

- 引用类型
- 或实现了某个接口
- 或有默认构造函数

✅ 必须自定义泛型委托

```
delegate string MyFormatter<T>(T value)
    where T : class, new();
```

这样你就保证了：

```
MyFormatter<string> f1 = ...; // ✅
MyFormatter<int> f2 = ...;    // ❌ 编译期直接拦住
```

📌 **这是 Action / Func 绝对做不到的**

六、把这段话“翻译”回你的代码

你现在的代码：

```
internal delegate void Feedback(Int32 value);
```

如果有一天你想改成这样👇

```
internal delegate void Feedback(
    Int32 start = 0,
    params Int32[] values
);
```

或者：

```
    internal delegate TResult Feedback<T>(T value)
        where T : IComparable<T>;
```

👉 **你就必须用自定义 delegate**
👉 **不能用 `Action<int>` / `Func<int,int>` 替代**

七、为什么 .NET 不把这些功能加进 `Action/Func`？

这是**设计取舍**：

- `Action/Func` 是 **“最小公分母”**
- 它们要保证：
  - 所有 .NET 版本兼容
  - 所有语言（C# / VB / F#）都能用
- 一旦加了 `params / default / 约束`，就会：
  - 爆炸式增长（几十上百个重载）
  - 破坏 ABI / API 稳定性

所以 Richters 的结论是：

> **“通用委托负责 80% 场景，剩下的 20%，你自己定义 delegate。”**

八、一句话终极总结

> **`Action/Func` 是“语法糖工厂产品”，功能固定；**
> **自定义 `delegate` 是“手工打造”，支持 `params`、默认值、泛型约束。**

如果你愿意，下一步我可以帮你把 **“什么时候该用 Action/Func，什么时候必须自定义 delegate”** 整理成一张**决策表**，这在真实项目里非常实用。你要吗？

用户:

这一段不太懂，请结合例子解释：

“重要提示 毫无疑问，C# 的 lambda 表达式功能很容易被程序员滥用。我开始使用 lambda 表达式时，绝对是花了一些时间来熟悉它的。毕竟，你在一个方法中写的代码实际不在这个方法中。除了有违直觉，还使调试和单步执行变得比较有挑战性。但事实上，Visual Studio 调试器还是非常不错的。我对自己源代码中的 lambda 表达式进行单步测试时，它处理得相当好。

我为自己设定了一个规则：如果需要在回到方法中包含 3 行以上的代码，就不使用 lambda 表达式。相反，我会手动写一个方法，并为其分配自己的名称。但如果使用得当，匿名方法确实能显著提高开发人员的效率和代码的可维护性。在以下代码中，使用 lambda 表达式感觉非常自然。没有它们，这样的代码会很难写、读和维护。”

元宝:

这段话是 Richter 的**实战经验之谈**，讲的是 **lambda 的“爽”和“坑”**。我结合你已有的 `Counter / Feedback` 代码，把这段话“翻译”成你能直接感受到的例子。

一、先说核心感受：**lambda 是“隐形的方法”**

你之前写的代码是这样的：

```
private static void FeedbackToConsole(Int32 value) {
    Console.WriteLine("Item=" + value);
}
```

这是一个**真方法**：

- 有名字
- 在类中
- 调试时能看见
- 调用栈里能找到

换成 **lambda** 后：

```
Counter(1, 3, val => Console.WriteLine("Item=" + val));
```

你以为代码在这里，其实不在。

编译器会悄悄生成一个**匿名类 + 匿名方法**：

```
[CompilerGenerated]
private static class <>c {
    public static void <Main>b__0_0(Int32 val) {
        Console.WriteLine("Item=" + val);
    }
}
```

👉 **真正的代码在 `<Main>b__0_0` 里**
👉 **不在你写 lambda 的那个方法里**

这就是 Richter 说的：

> “你在一个方法中写的代码实际不在这个方法中。”

二、为什么说“有违直觉 + 调试有挑战”？

1️⃣ 调用栈不一样（直观感受）

假设 lambda 里出错了：

```
Counter(1, 3, val => {
    int x = 10;
    int y = x / (val - 2); // val=2 时抛异常
    Console.WriteLine(y);
});
```

异常调用栈可能是：

```
at Program.<Main>b__0_0(Int32 val)
at Program.Counter(Int32 from, Int32 to, Feedback fb)
at Program.Main()
```

你看到的是 `<Main>b__0_0`，不是你写的 lambda 行号（虽然 VS 能映射回去）。

2️⃣ 变量“神秘消失 / 出现”

```
int factor = 10;

Counter(1, 3, val => {
    Console.WriteLine(val * factor);
});
```

调试时你会发现：

- `factor` **不在当前栈帧**
- 它在编译器生成的闭包类里：`<>c__DisplayClass1.factor`

👉 这就是“有违直觉”的来源

三、Richter 的“3 行规则”（非常重要）

他说：

> **“如果需要在 lambda 中包含 3 行以上的代码，就不使用 lambda。”**

❌ 超过 3 行，立刻变难读

```
Counter(1, 3, val => {
    var sw = new StreamWriter("log.txt", true);
    sw.WriteLine($"Processing {val}");
```

```
        sw.Flush();
        sw.Dispose();
        Console.WriteLine(val);
    });
```

问题：

- 逻辑复杂
- 异常处理不明显
- 调试困难
- 方法名缺失（语义不清）

✅ 改成普通方法，立刻清晰

```
    private static void LogAndPrint(int val) {
        using var sw = new StreamWriter("log.txt", true);
        sw.WriteLine($"Processing {val}");
        Console.WriteLine(val);
    }

    Counter(1, 3, new Feedback(LogAndPrint));
```

优点：

- 有名字（语义明确）
- 可复用
- 可单独测试
- 调试友好

四、那 **lambda** 什么时候“用起来很自然”？

Richter 说：**“以下代码中，使用 lambda 感觉非常自然。”**

典型场景就是 **“一眼就能看懂意图”的小逻辑。**

✅ **LINQ /** 排序 **/** 过滤

```
    var list = new List<int> { 1, 2, 3 };
    var even = list.Where(x => x % 2 == 0);
```

```
    Array.Sort(arr, (x, y) => x.CompareTo(y));
```

这些 lambda：

- 一行
- 无副作用
- 纯逻辑
- 一看就懂

👉 **这是 lambda 的主场**

✅ 在你现有代码里的“自然用法”

```
Counter(1, 3, val => Console.WriteLine(val));
```

这完全符合 Richter 的风格：

- 一行
- 目的明确
- 不捕获复杂状态

五、一句话总结 **Richter** 的潜台词

> **lambda 是“语法糖”，不是“随便写”。**
> **它适合表达“做什么”，不适合表达“怎么做”。**
> **一旦逻辑开始复杂，就还给“方法”。**

六、给你一个可执行的“自查清单”

写 lambda 前，心里默念三问：

1. **超过 3 行了吗？** → 不用 lambda
2. **需要捕获外部变量吗？** → 谨慎
3. **别人一周后能一眼看懂吗？** → 不能就拆方法

七、把这段话和 **CLR** 连起来（进阶）

你之前问过“包装器”，现在可以连起来了：

- **lambda → 编译器生成闭包类**
- **闭包类 → 包含一个方法**
- **该方法 → 被包装进委托对象**

- **委托 → `_target` 指向闭包实例**

也就是说：

> **lambda = 闭包类 + 方法 + 委托包装器**

这也是为什么 lambda 调试“有挑战”——你调试的不是你写的代码，而是**编译器生成的代码。**

如果你愿意，下一步我可以帮你把 **“lambda → 闭包类 → IL → 委托”** 的完整生成过程，用你这个 `Counter` 示例画一条**从源码到 CLR 的执行链**，那基本就把第 17 章后半段吃透了。你要继续吗？

用户:

文章中的这个例子不太懂：

using System;

using System.Reflection;

using System.IO;

using System.Linq;

// 下面是一些不同的委托定义

internal delegate Object TwoInt32s(Int32 n1, Int32 n2);

internal delegate Object OneString(String s1);

public static class DelegateReflection {

```
    public static void Main(String[] args) {

        if (args.Length < 2) {

            String usage =

                @"Usage:" +

                "{0} delType methodName [Arg1] [Arg2]" +

                "{0}    where delType must be TwoInt32s or OneString" +

                "{0}    if delType is TwoInt32s, methodName must be Add or Subtracr" +

                "{0}    if delType is OneString, methodName must be NumChars or Reverse" +

                "{0}" +

                "{0}Examples:" +

                "{0}    {1} TwoInt32s Add 123 321" +
```

```
            "{0}    {1} TwoInt32s Subtract 123 321" +
            "{0}    {1} OneString NumChars \"Hello there\"" +
            "{0}    {1} OneString Reverse \"Hello there\"";
        Console.WriteLine(usage, Environment.NewLine);
        return;
    }

    // 将 delType 实参转换为委托类型
    Type delType = Type.GetType(args[0]);
    if (delType == null) {
        Console.WriteLine("Invalid delType argument: " + args[0]);
        return;
    }

    Delegate d;
    try {
        // 将 Arg1 实参转换为方法
        MethodInfo mi = typeof(DelegateReflection).GetTypeInfo().GetDeclaredMethod(args
[1]);

        // 创建包装了静态方法的委托对象
        d = mi.CreateDelegate(delType);
    }
```

```
catch (ArgumentException) {

    Console.WriteLine("Invalid methodName argument: " + args[1]);

    return;

}



// 创建一个数组，其中只包含要通过委托对象传给方法的参数

Object[] callbackArgs = new Object[args.Length - 2];



if (d.GetType() == typeof(TwoInt32s)) {

    try {

        // 将 String 类型的参数转换为 Int32 类型的参数

        for (Int32 a = 0; a < args.Length; a++)

            callbackArgs[a - 2] = Int32.Parse(args[a]);

    }

    catch (FormatException) {

        Console.WriteLine("Parameters must be integers.");

        return;

    }

}



if (d.GetType() == typeof(OneString)) {

    // 只复制 String 参数
```

```
            Array.Copy(args, 2, callbackArgs, 0, callbackArgs.Length);

        }



        try {

            // 调用委托并显示结果

            Object result = d.DynamicInvoke(callbackArgs);

            Console.WriteLine("Result = " + result);

        }

        catch (TargetParameterCountException) {

            Console.WriteLine("Incorrect number of parameters specified.");

        }

    }



    // 这个回调方法获取 2 个 Int32 参数

    private static Object Add(Int32 n1, Int32 n2) {

        return n1 + n2;

    }



    // 这个回调方法获取 2 个 Int32 参数

    private static Object Subtract(Int32 n1, Int32 n2) {

        return n1 - n2;

    }
```

```
    // 这个回调方法获取 1 个 String 参数
    private static Object NumChars(String s1) {
        return s1.Length;
    }


    // 这个回调方法获取 1 个 String 参数
    private static Object Reverse(String s1) {
        return new String(s1.Reverse().ToArray());
    }
```

}

元宝:

这段代码是 **“委托 + 反射”的典型示例**，也是 Richter 用来展示 **“委托是运行时可以动态构造的回调机制”** 的关键例子。

我把它拆成 **“这段代码在干什么 → 每一步 CLR 发生了什么 → 为什么不用普通调用 → 一个最小执行流程”**，你跟着走一遍，基本就通了。

一、这段代码“整体在干什么？”（一句话版）

> **在运行时，根据命令行输入的“委托类型 + 方法名 + 参数”，动态创建一个委托对象，并用 `DynamicInvoke` 调用它。**

换句话说：

> **方法名、参数个数、参数类型，全部在运行前都不知道，全是运行时决定的。**

这是普通 `Add(1, 2)` 做不到的。

二、先跑一遍“正常用法”（建立直觉）

命令行输入：

```
    DelegateReflection.exe TwoInt32s Add 123 321
```

程序最终做的是：

```
TwoInt32s d = new TwoInt32s(DelegateReflection.Add);
object result = d.DynamicInvoke(123, 321);
Console.WriteLine(result); // 444
```

但你**并没有在代码里写** `Add` **或** `TwoInt32s`，一切都是运行时拼出来的。

三、代码逐段拆解（重点来了）

---

① 两个委托类型（不是方法）

```
internal delegate Object TwoInt32s(Int32 n1, Int32 n2);
internal delegate Object OneString(String s1);
```

这是重点：

- **委托 ≠ 方法**
- 它们是“方法签名模板”
- 后面反射时要**按签名匹配**

---

② 根据字符串拿到“委托类型”

```
Type delType = Type.GetType(args[0]);
```

如果输入的是 `"TwoInt32s"`：

```
delType == typeof(TwoInt32s)
```

👉 **这是反射的第一步：从字符串 → Type**

---

③ 根据字符串拿到“方法”

```
MethodInfo mi = typeof(DelegateReflection)
    .GetTypeInfo()
    .GetDeclaredMethod(args[1]);
```

如果输入的是 `"Add"`：

```
mi == MethodInfo of DelegateReflection.Add
```

👉 **这是反射的第二步：从字符串 → MethodInfo**

---

④ 关键一步：`CreateDelegate`

```
d = mi.CreateDelegate(delType);
```

这是**整段代码的灵魂。**

它在干什么？

- 检查：
  - `Add` 的签名是否匹配 `TwoInt32s`
    - 参数：两个 `Int32`
    - 返回值：`Object`
- 创建一个 **委托对象**
- 内部设置：
  - `_target = null`（静态方法）
  - `_methodPtr = Add`
  - `_target` + `_methodPtr` → 包装成委托

等价于（编译期写法）：

```
TwoInt32s d = new TwoInt32s(DelegateReflection.Add);
```

但区别是：

写法	时机
`new TwoInt32s(Add)`	编译期绑定
`CreateDelegate`	运行期绑定

---

⑤ 准备参数数组

```
Object[] callbackArgs = new Object[args.Length - 2];
```

为什么是 `args.Length - 2`？

- `args[0]` → 委托类型

- `args[1]` → 方法名
- 剩下的是参数

例如：

```
TwoInt32s Add 123 321
↑0         ↑1   ↑2  ↑3
```

参数就是 `[123, 321]`

---

⑥ 根据委托类型“解释参数”

如果是 **`TwoInt32s`**

```
callbackArgs[a - 2] = Int32.Parse(args[a]);
```

因为命令行参数是 `string`，但方法要的是 `int`。

如果是 **`OneString`**

```
Array.Copy(args, 2, callbackArgs, 0, callbackArgs.Length);
```

字符串直接传。

👉 **这是“类型适配”的过程**

---

⑦ 灵魂调用：`DynamicInvoke`

```
Object result = d.DynamicInvoke(callbackArgs);
```

为什么不用 **`d(123, 321)`**？

因为：

- 编译期 **不知道 `d` 是什么委托**
- 不知道参数个数
- 不知道参数类型

`DynamicInvoke` 的特点是：

- ✅ 参数用 `object[]`
- ✅ 运行时检查签名
- ✅ 自动装箱 / 拆箱
- ❌ 慢（反射 + 委托）

它内部大致做了：

1. 检查 `callbackArgs` 是否匹配委托签名
2. 必要时做类型转换
3. 调用 `Invoke`
4. 返回 `object`

四、为什么这个例子“非用反射不可”？

普通写法（做不到）

```
// 编译期就必须确定
TwoInt32s d = Add;
d(123, 321);
```

但这个例子可以做到：

输入	实际调用
`TwoInt32s Add`	`Add(123,321)`
`TwoInt32s Subtract`	`Subtract(123,321)`
`OneString Reverse`	`Reverse("hello")`

👉 **行为完全由输入决定**

这是：

- 插件系统
- 脚本引擎
- 规则引擎
- ASP.NET MVC 路由 / 过滤器
- Unity / Autofac 这类框架

的**核心机制**。

五、把“委托是包装器”再套进来一次（关键）

```
d = mi.CreateDelegate(delType);
```

这一步的本质是：

> **把 MethodInfo（方法元数据）包进一个委托对象里**

委托对象里最终是：

字段	值
`_target`	`null`（静态方法）
`_methodPtr`	`Add / Subtract / NumChars / Reverse`
`_invocationList`	`null`

`DynamicInvoke` 只是：

> **通过这个包装器，间接调用方法**

六、一个“最小心智模型”（建议记住）

```
字符串
 ↓ Type.GetType
Type（委托类型）
 ↓ GetDeclaredMethod
MethodInfo（方法）
 ↓ CreateDelegate
委托对象（包装器）
 ↓ DynamicInvoke
真正的方法执行
```

七、**Richter** 为什么放这个例子？

他在告诉你三件事：

1. **委托不是语法糖，是 CLR 一级的概念**
2. **委托可以在运行时动态构造**
3. **`DynamicInvoke` 是“最慢但最灵活”的调用方式**

同时也暗示：

> **日常业务代码不要用 `DynamicInvoke`**
> **框架 / 基础设施代码才会用它**

八、一句话终极总结

> **这个例子展示了：委托是“可以在运行时组装的回调包装器”，而反射负责“找到方法”，`CreateDelegate` 负责“打包”，`DynamicInvoke` 负责“兜底调用”。**

如果你愿意，下一步我可以把 `CreateDelegate` **和** `new Delegate(...)` **在 CLR 内部的差异** 讲一下，或者帮你对比 `Invoke` **vs** `DynamicInvoke` **的性能差距** ——这在面试和实际项目中都很常见。你更想继续哪一块？

（注：文档部分内容可能由 AI 生成）